NGO Impact Reporter

A full-stack web application that allows NGOs to submit monthly impact reports and enables admins to view aggregated data through a protected dashboard. Supports single report submission, bulk CSV upload with background processing, and real-time job progress tracking.

Table of Contents

- Tech Stack
- Getting Started
- Environment Variables
- API Reference
- CSV Format
- Writeup

Tech Stack

Layer	Technology
Backend	Node.js, Express, PostgreSQL (pg), JWT
Frontend	React 18, TypeScript, Vite, MUI, React Query
Background Jobs	In-process async queue (setImmediate)

Getting Started

You'll need **Node.js** and a **PostgreSQL** instance running locally.

Via GitHub

```
git clone https://github.com/Manisha171/ngo-reporter.git
cd ngo-reporter
```

Via zip

Extract the zip and open the folder in your terminal.

Backend

```
cd backend
npm install
cp .env.example .env
npm run dev
```

> Runs on `http://localhost:4000`

Frontend

```
cd frontend
npm install
npm run dev
```

> Runs on `http://localhost:5173`

Environment Variables

Create a `.env` file inside the `backend` folder using `.env.example` as a template:

Variable	Description	Default
`DATABASE_URL`	PostgreSQL connection string	`postgresql://localhost:5432/ngo_reporter`
`JWT_SECRET`	Secret key for JWT signing	—

`ADMIN_USER`	Admin login username	`admin`
`ADMIN_PASS`	Admin login password	`admin123`
`PORT`	Backend server port	`4000`
`CLIENT_URL`	Frontend origin for CORS	`http://localhost:5173`

> Database tables (`reports`, `jobs`) are created automatically on first run.

API Reference

Authentication

Method	Endpoint	Description
`POST`	`/auth/admin`	Admin login — returns JWT

Reports

Method	Endpoint	Auth	Description
`POST`	`/report`	No	Submit a single report
`POST`	`/reports/upload`	No	Upload CSV file — returns `job_id`
`GET`	`/job-status/:job_id`	No	Poll CSV processing status
`GET`	`/dashboard?month=YYYY-MM`	Admin JWT	Aggregated stats for a month

Dashboard also supports `&region=North` and `&ngo_id=NGO-001` query params for filtering.

Sample Requests

Submit a single report

```
curl -X POST http://localhost:4000/report \
  -H "Content-Type: application/json" \
  -d '{
    "ngo_id": "NGO-001",
    "month": "2024-03",
    "people_helped": 120,
    "events_conducted": 5,
    "funds_utilized": 45000,
    "region": "North"
  }'
```

Upload a CSV

```
curl -X POST http://localhost:4000/reports/upload \
  -F "file=@reports.csv"

# Response: { "job_id": "uuid" }
```

Poll job status

```
curl http://localhost:4000/job-status/<job_id>

# Response:
# {
#   "status": "done",
#   "total": 15,
#   "processed": 13,
#   "failed": 2,
#   "errors": [{ "row": 9, "reason": "Missing required fields" }]
# }
```

Dashboard

```
curl "http://localhost:4000/dashboard?month=2024-03&region=North" \
  -H "Authorization: Bearer <admin_token>"
```

CSV Format

```
ngo_id,month,people_helped,events_conducted,funds_utilized,region
NGO-001,2024-03,120,5,45000,North
NGO-002,2024-03,80,3,30000,South
```

- `region` is optional — rows without it are accepted
- Rows with missing required fields or invalid numbers are marked as failed
- Duplicate entries (same `ngo_id` + `month`) are silently skipped

Writeup

Approach

The main challenge was async CSV processing. When a file is uploaded, the backend immediately returns a `job_id` and hands off processing via `setImmediate`. The frontend polls `/job-status/:job_id` every 1.5 seconds and updates a progress bar in real time. This keeps the upload response fast regardless of file size.

Idempotency is enforced at the database level — the `reports` table has a `UNIQUE(ngo_id, month)` constraint and all inserts use `ON CONFLICT DO NOTHING`. Re-uploading the same CSV or resubmitting the same form won't create duplicates.

Partial failures are handled per row. Each failed row is logged with its row number and reason. The job completes even if some rows fail, and the UI shows exactly which rows failed and why. Failed rows are retried up to 3 times before being marked as failed.

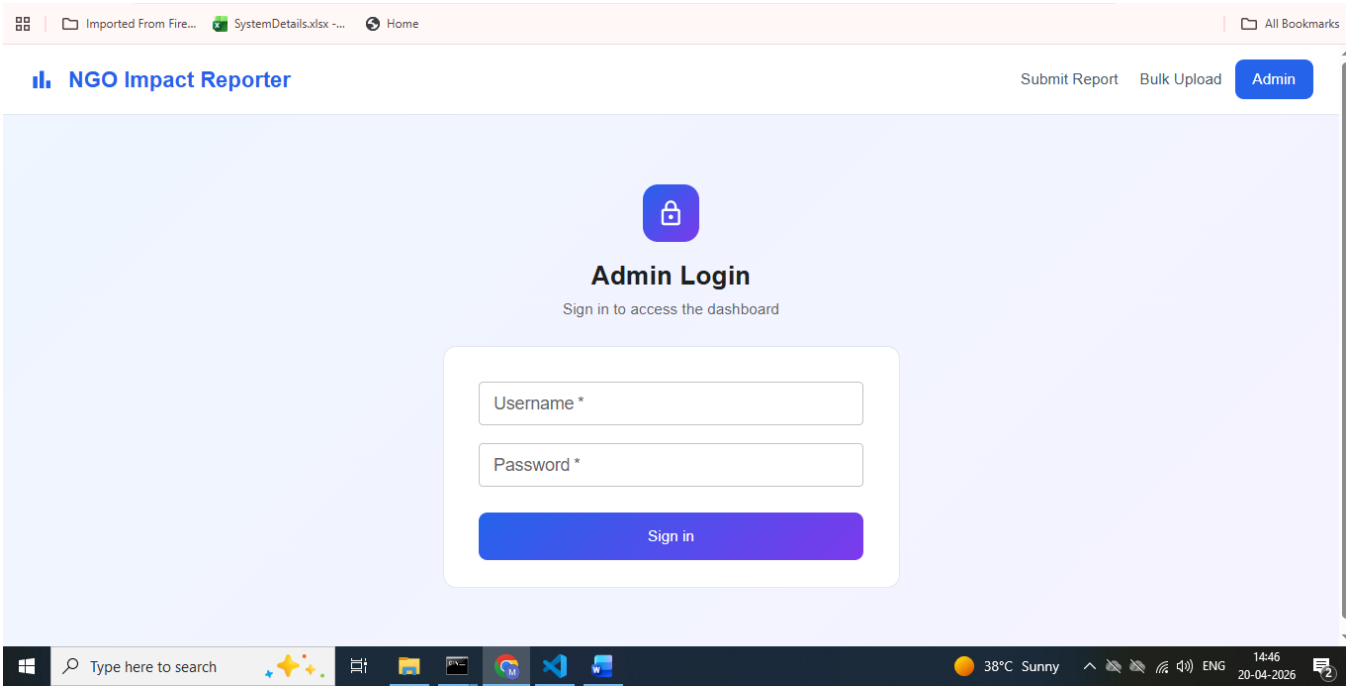
Where AI Tools Helped

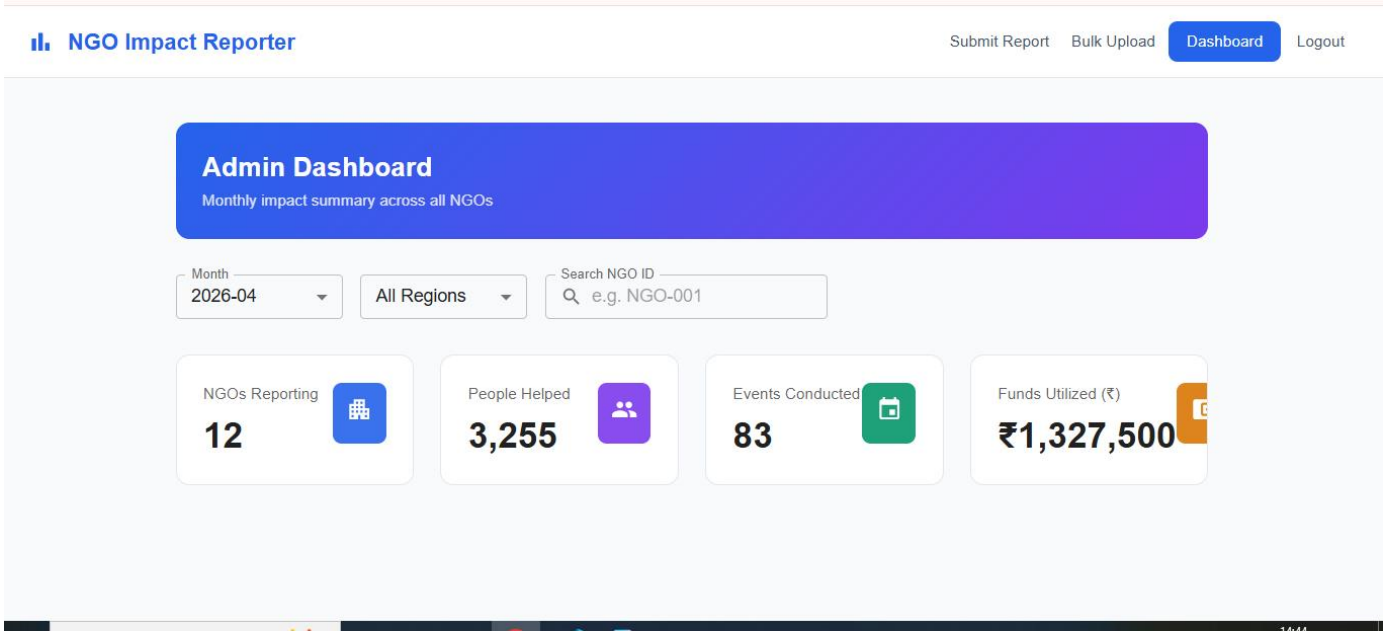
Used GitHub Copilot occasionally for repetitive boilerplate — things like MUI component structure and TypeScript interface definitions. Everything else was written and reasoned through manually.

What I'd Improve in Production

- Replace the in-process queue with **Bull + Redis** so jobs survive server restarts
- Add **pagination** to a reports list view on the dashboard
- **Rate limiting** on the `/report` and `/reports/upload` endpoints
- **Refresh tokens** so admin sessions don't expire mid-use
- **Structured logging** with Winston or Pino instead of console.log

Screenshot




NGO Impact Reporter
Submit Report
Bulk Upload
Dashboard
Logout
Admin Dashboard
Monthly impact summary across all NGOs
Month
2026-04
All Regions
Search NGO ID
e.g. NGO-001
NGOs Reporting
12
People Helped
3,255
Events Conducted
83
Funds Utilized (₹)
₹1,327,500